

CAHIER DES CHARGES

Spécification Technique et Pédagogique

Identity Document Forgery Detection

via DINOv2-Grounded Sparse VAE with Multi-Scale Localization

Etudiantes	Institution	Version	Encadrantes
Wissal KARABAKA & Ghada TLILI	ENET'COM Sfax — IDSD	Juin 2026	Mme Sonda AMMAR Mme Mariem Regaieg

Vue d'ensemble du pipeline

Ce document est un cahier des charges technique et pédagogique destiné à guider l'implémentation complète du pipeline de détection de falsification de documents d'identité. Chaque étape est décrite avec : son rôle précis dans l'architecture globale, les concepts théoriques sous-jacents, les entrées et sorties attendues, les paramètres à implémenter avec justification, les extraits de code de référence, et les critères de validation.

Contexte et motivation

La détection de falsification de documents d'identité (CNI, passeports, permis de conduire) est un problème critique pour les systèmes KYC (Know Your Customer), le contrôle aux frontières, et les institutions financières. Le défi principal est la rareté des données : on dispose de certains documents authentiques, mais les exemples de forgeries étiquetées sont rares, juridiquement sensibles, et ne couvrent pas tous les types de manipulation possibles.

Notre approche répond à ce défi en formulant le problème comme de la détection d'anomalies non supervisée : le modèle apprend exclusivement la distribution des documents authentiques, et tout document qui s'en écarte significativement est classifié comme forgé. Aucun exemple de forgerie n'est nécessaire à l'entraînement.

Principe fondamental

Un document authentique → le modèle le reconstruit bien → erreur faible → score d'anomalie bas → AUTHENTIQUE

Un document forgé → le modèle échoue à le reconstruire → erreur élevée → score d'anomalie haut → FORGÉ

Architecture globale

Le pipeline complet enchaîne 7 étapes qui transforment une image de document en une décision binaire (authentique / forgé) accompagnée d'une carte de localisation de la forgerie :

1	2	3	4	5	6	7
DINOv2	Projection	Sparse Latent	Decoder	Loss	Heatmap	Score

#	Étape	Entrée	Sortie	Paramètres clés
1	DINOv2 Extraction	Image 224×224 RGB	Patch tokens (256×768)	Frozen ViT-B/14
2	Projection Head	Tokens (256×768)	$\mu, \log \sigma^2$ (64-dim)	3 couches Linear+GELU
3	Sparse Latent	$\mu, \log \sigma^2$	z sparse (64-dim)	K=16, $\beta=4$
4	Multi-Scale Decoder	z (64-dim)	$\hat{x}$ à 3 échelles	FPN 16/32/64px
5	Reconstruction Loss	x original, $\hat{x}$	Scalaire loss	$\alpha=0.8, \gamma=0.2$

#	Étape	Entrée	Sortie	Paramètres clés
6	Anomaly Heatmap	Erreurs multi-échelle	Carte H (64×64)	Fusion pondérée
7	Anomaly Score	H, KL, z norm	Score scalaire S	Seuil θ calibré

01

DINOv2 Feature Extraction

Backbone visuel pré-entraîné — frozen — extracteur de features riches

1.1 Pourquoi DINOv2 ? — Compréhension conceptuelle

Un Vision Transformer (ViT) entraîné from scratch nécessite des centaines de milliers d'images pour apprendre à représenter correctement le contenu visuel. MIDV-2020 ne contient que 1 000 images : entraîner un ViT from scratch provoquerait un overfitting massif.

DINOv2 est un ViT pré-entraîné par Meta AI sur LVD-142M (142 millions d'images) via une objective self-supervised (DINO = self-Distillation with NO labels). Il a appris des représentations générales particulièrement adaptées à notre cas :

- Il est sensible aux textures locales fines (utile pour les microprints, les guilloches)
- Ses patch tokens sont localement discriminants : chaque token représente une zone 14×14px du document
- Il généralise bien à des domaines visuels variés, y compris les documents
- Il est disponible en open-source et s'intègre en 2 lignes de code PyTorch

i Info

Comparaison avec les alternatives : CLIP (orienté image-texte, moins discriminant localement), ViT supervised ImageNet (moins robuste aux textures fines), CNN ResNet (pas de tokens locaux, moins adapté à la heatmap). DINOv2 est le meilleur choix pour ce pipeline.

1.2 Fonctionnement interne du ViT

Un Vision Transformer découpe l'image en patches carrés de taille fixe, transforme chaque patch en un vecteur (token), et applique des couches d'attention multi-têtes pour capturer les relations entre patches. La sortie est un ensemble de tokens enrichis par le contexte global de l'image.

Pour DINOv2 ViT-B/14 :

Paramètre	Valeur / Spécification	Justification
Taille de patch	14 × 14 pixels	Résolution fine pour forgeries localisées
Résolution d'entrée	224 × 224 pixels	Standard DINOv2
Nombre de patches	$(224/14)^2 = 256$ patches	Grille 16×16 de tokens
Token spécial [CLS]	1 token global additionnel	Représentation de l'image entière
Dimension par token	768 dimensions (ViT-B)	Représentation riche
Sortie totale	257 tokens × 768 dims	256 patches + 1 CLS
Nombre de couches	12 transformer blocks	Deep representation
Paramètres totaux	~86 millions (frozen)	Aucun gradient calculé

1.3 Entrées / Sorties détaillées

Tenseur	Shape	Dtype	Description
Entrée : image	(B, 3, 224, 224)	float32	Batch d'images normalisées
Sortie : patch_tokens	(B, 256, 768)	float32	Un vecteur 768-dim par patch 14×14px
Sortie : cls_token	(B, 768)	float32	Représentation globale de l'image

1.4 Critères de validation — Comment vérifier que ça marche

- ▶ Visualisation PCA : projeter les patch_tokens en 3 composantes (PCA) et colorier par zone document (texte, photo, fond). Les zones doivent former des clusters distincts.
- ▶ Visualisation t-SNE : les cls_tokens de différents types de documents (carte d'identité albanaise vs passeport grec) doivent former des groupes séparés.
- ▶ Test de cohérence : deux augmentations légères du même document doivent produire des features très proches (cosine similarity > 0.90).
- ▶ Temps : l'extraction de 1 image doit prendre < 5ms sur GPU T4 (DINOv2 ViT-B/14 est optimisé).

⚠ Attention

Ne pas appliquer d'augmentations agressives (RandomCrop aléatoire, forte rotation, flip vertical) : les documents ont une orientation fixe et une mise en page structurée. Une augmentation trop forte apprendrait des invariances inappropriées.

02

Projection Head

Mapping des features DINOv2 vers les paramètres de la distribution latente VAE

2.1 Rôle et principe

DINOv2 produit 256 tokens de 768 dimensions — soit 196 608 valeurs par image. L'espace latent du VAE ne fait que 64 dimensions. Le Projection Head est le réseau qui apprend à comprimer cette information vers deux vecteurs : μ (moyenne) et $\log \sigma^2$ (log-variance), qui définissent une distribution gaussienne dans l'espace latent.

C'est la seule partie entraînable côté encodeur. DINOv2 est frozen — il fournit des features fixes et riches. Le Projection Head apprend comment réorganiser ces features pour optimiser la reconstruction de documents authentiques.

Analogie pédagogique

DINOv2 est comme un expert en vision qui décrit chaque image en 256 rapports détaillés. Le Projection Head est le résumé exécutif : il synthétise ces rapports en 64 caractéristiques essentielles, tout en quantifiant son incertitude (σ^2) sur chaque caractéristique.

2.2 Pourquoi μ et $\log \sigma^2$? — Base théorique VAE

Un Autoencoder standard encode chaque image en un point fixe z dans l'espace latent. Un VAE encode chaque image en une distribution : $z \sim N(\mu, \sigma^2)$. Cette différence est fondamentale pour la détection d'anomalies :

- ▶ Un document authentique → le modèle est confiant → σ^2 faible (distribution étroite)
- ▶ Un document forgé → le modèle est incertain → σ^2 élevé (distribution large)
- ▶ La variance σ^2 devient donc un signal d'anomalie en soi, en complément de l'erreur de reconstruction

Le $\log \sigma^2$ (log de la variance) est utilisé plutôt que σ^2 directement car il peut prendre des valeurs négatives (variance < 1) et des valeurs positives (variance > 1), ce qui est numériquement plus stable pour l'optimisation.

2.3 Architecture détaillée

Paramètre	Valeur / Spécification	Justification
Agrégation spatiale	Mean pooling : $(B, 256, 768) \rightarrow (B, 768)$	Moyenne des 256 patch tokens
Couche FC 1	Linear(768, 512) + LayerNorm(512) + GELU	Réduction dimensionnelle progressive
Dropout 1	Dropout($p=0.1$)	Régularisation légère
Couche FC 2	Linear(512, 256) + LayerNorm(256) + GELU	Compression supplémentaire
Dropout 2	Dropout($p=0.1$)	Régularisation
Tête μ	Linear(256, 64) — aucune activation	μ peut être $\in \mathbb{R}$

Paramètre	Valeur / Spécification	Justification
Tête log σ^2	Linear(256, 64) + Clamp(-4, 4)	Clamp évite variance explosive
Sortie	$(\mu, \log \sigma^2)$ — deux tenseurs (B, 64)	Paramètres de $q(z x)$

2.4 Reparametrization Trick — implémentation

Pour pouvoir calculer les gradients à travers l'opération d'échantillonnage stochastique, on utilise le reparametrization trick : au lieu de sampler z directement depuis $N(\mu, \sigma^2)$, on sample ε depuis $N(0, I)$ et on calcule $z = \mu + \sigma \odot \varepsilon$. Les gradients peuvent ainsi remonter à travers μ et σ .

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim N(0, I)$$

où $\sigma = \exp(0.5 \times \log \sigma^2)$

```
def reparameterize(mu, logvar):
    """
    Reparametrization trick : z = mu + sigma * epsilon
    Différentiable par rapport à mu et logvar.
    """
    if self.training:
        std = torch.exp(0.5 * logvar) #  $\sigma = \exp(0.5 \times \log \sigma^2)$ 
        eps = torch.randn_like(std)   #  $\varepsilon \sim N(0, I)$ , même shape que std
        return mu + std * eps         # (B, 64)
    else:
        # À l'inférence : moyenne de L=10 échantillons pour stabilité
        std = torch.exp(0.5 * logvar)
        z_samples = [mu + std * torch.randn_like(std) for _ in range(10)]
        return torch.stack(z_samples).mean(dim=0)
```

2.6 Critères de validation

- ▶ Les distributions $q(z|x)$ pour des documents authentiques doivent être proches de $N(0, I)$: vérifier que $\mu_{\text{moyen}} \approx 0$ et $\sigma^2_{\text{moyen}} \approx 1$ sur le val set.
- ▶ La KL divergence moyenne sur les authentiques doit converger entre 2 et 10 nats après entraînement complet.
- ▶ Aucune dimension ne doit être en posterior collapse : vérifier que $\text{Var}(\mu_i) > 0.01$ pour toutes les dimensions $i \in [0, 63]$.

03

Sparse Latent Space

Top-K hard sparsity + β -KL regularization — cœur de la contribution

3.1 Problème résolu — Pourquoi la sparsité

Un VAE standard utilise toutes ses 64 dimensions latentes simultanément. Cela pose deux problèmes pour la détection d'anomalies :

- Redondance : plusieurs dimensions encodent la même information → le modèle reconstruit bien même les anomalies
- Manque de sélectivité : les variations anodines (luminosité, orientation légère) occupent autant de dimensions latentes que les caractéristiques structurales des documents → sensibilité diluée

La sparsité force le modèle à n'utiliser que les K dimensions les plus informatives. Les dimensions redondantes sont mises à zéro. **Résultat** : le modèle ne peut reconstruire que ce qu'il a appris de ses K dimensions actives — si un document forgé présente une structure légèrement différente, la reconstruction échoue plus nettement.

3.2 Top-K Hard Sparsity — Mécanisme

Après le reparametrization trick, on obtient $z \in \mathbb{R}^{64}$. On applique un masque qui met à zéro toutes les dimensions sauf les K dont $|z_i|$ est le plus grand.

```
mask_i = 1 si |z_i| ∈ top-K(|z|)
mask_i = 0 sinon
z_sparse = z ⊙ mask
```

Le problème du top-K pour l'entraînement : l'opération argmax n'est pas différentiable. On utilise le straight-through estimator : en forward, on applique le masque (0 ou 1) ; en backward, on laisse passer le gradient comme si le masque n'existait pas.

```
def top_k_sparsity(z, k=16):
    """
    Applique une sparsité Top-K sur z.
    Seules les K dimensions de plus forte magnitude sont conservées.
    Gradient : straight-through estimator.
    """
    # |z| pour trouver les K plus grandes valeurs
    z_abs = z.abs() # (B, 64)

    # Top-K threshold : valeur de la K-ième plus grande magnitude
    threshold, _ = z_abs.topk(k, dim=1) # (B, K)
    threshold = threshold[:, -1:] # (B, 1) — la plus petite des K
    plus grandes

    # Masque binaire : 1 pour les K dims actives, 0 pour les autres
    mask = (z_abs >= threshold).float() # (B, 64) — 0 ou 1

    # Straight-through : forward applique le masque,
    # backward laisse passer le gradient entier
    z_sparse = z * mask + z.detach() * (1 - mask) - z.detach() * (1 -
mask)
```



```
# Simplification : z_sparse = z * mask (suffisant pour la pratique)
z_sparse = z * mask

return z_sparse, mask
```

3.3 β -KL Regularization

La β -VAE (Higgins et al., 2017) ajoute un poids $\beta > 1$ sur le terme KL de la loss. Cela force la distribution latente à rester proche de la prior $N(0, I)$, ce qui améliore la structure de l'espace latent et facilite la détection d'anomalies : un document forgé dévie de $N(0, I) \rightarrow$ KL élevée $\rightarrow$ signal d'anomalie.

$KL(q(z|x) || p(z)) = -0.5 \times \sum_i (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2)$
 Terme KL dans la loss = $\beta \times KL(q||p)$, avec $\beta > 1$

Paramètre	Valeur / Spécification	Justification
Valeur de β	$\beta = 4$ (défaut)	Ablation sur $\beta \in \{1, 2, 4, 8\}$
Valeur de K	$K = 16$ (25% de 64)	Ablation sur $K \in \{8, 16, 24, 32\}$
Warm-up β	Annealing 0 $\rightarrow$ 4 sur 20 epochs	Évite le posterior collapse en début
Application	Top-K après reparametrization	Sparsité sur z, pas sur μ

3.4 Analyse de l'espace latent (LEA — Latent Efficiency Analysis)

Après chaque epoch de validation, calculer et logger ces métriques pour diagnostiquer l'état de l'espace latent :

Métrique	Calcul	Interprétation	Valeur cible
Active ratio	% de samples avec $ z_i > 0.01$ pour chaque dim i	0.3–0.7 = sparsité saine	$0.3 < r \leq 0.7$
Sparsity rate	% de dims actives moyennées sur un batch	Mesure globale de sparsité	~25% ($K=16/64$)
Entropy latente	$H = -\sum p_i \log p_i$ sur les activations	Diversité de l'utilisation	Modérée
Variance/dim	$\text{Var}(z_i)$ sur le val set, par dimension	Dimensions mortes si $\text{Var} < 0.001$	Toutes > 0.001

i Info

Active ratio < 0.3 pour plusieurs dimensions $\rightarrow$ posterior collapse : augmenter l_r ou diminuer β . Active ratio > 0.9 pour toutes les dimensions $\rightarrow$ pas de sparsité effective : augmenter K ou réduire β .

3.5 Implémentation complète de la couche latente

```
class SparseLatentLayer(nn.Module):
    def __init__(self, latent_dim=64, k=16):
        super().__init__()
        self.k = k
        self.latent_dim = latent_dim

    def forward(self, mu, logvar):
        # 1. Reparametrization
        z = reparameterize(mu, logvar)          # (B, 64)

        # 2. Top-K sparsity
        z_sparse, mask = top_k_sparsity(z, self.k)  # (B, 64)

        # 3. KL divergence (calculée sur z original, pas z_sparse)
        kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(),
dim=1)
        kl_loss = kl_loss.mean()  # Moyenne sur le batch

        return z_sparse, kl_loss, mask

    def active_ratio(self, mask):
        """Fraction de dimensions actives – pour monitoring LEA"""
        return mask.float().mean().item()
```

04

Multi-Scale Decoder (FPN-style)

Reconstruction à 3 granularités alignées sur les forgeries de FMIDV-2022

4.1 Motivation — Pourquoi multi-échelle

FMIDV-2022 contient des forgeries de type copy-move à trois tailles : 16×16 pixels (très petites), 32×32 pixels (moyennes) et 64×64 pixels (grandes). Une forgery 16×16 représente moins de 1% des pixels d'une image 224×224 — un décodeur standard produisant une seule reconstruction à haute résolution va "noyer" l'erreur de reconstruction de ces petites zones dans la globalité de l'image.

Un décodeur multi-échelle reconstruit l'image à chaque résolution correspondante. L'erreur de reconstruction à l'échelle 16×16 est calculée sur une représentation 16×16 — la forgery occupe alors une fraction beaucoup plus large et est mieux détectée.

4.2 Architecture FPN (Feature Pyramid Network style)

L'architecture suit un décodeur pyramidal : on part d'une feature map 8×8 et on l'upscale progressivement par des ConvTranspose2d, en ajoutant des skip connections latérales à chaque niveau pour conserver les détails fins.

Paramètre	Valeur / Spécification	Justification
Entrée	<code>z_sparse (B, 64)</code>	Vecteur latent sparse de l'étape 3
Projection initiale	<code>Linear(64, 512) + GELU → view(B, 8, 8)</code>	8 feature maps 8×8
Niveau 1 (coarse)	<code>ConvTranspose2d(8→128, 4×4, stride=2) → (B,128,16,16) + BN + GELU</code>	Résolution 16×16
Tête coarse	<code>Conv2d(128, 3, 1×1) + Sigmoid → $\hat{x}_c$ (B,3,16,16)</code>	Reconstruction 16px
Niveau 2 (medium)	<code>ConvTranspose2d(128→64, 4×4, stride=2) → (B,64,32,32) + BN + GELU</code>	Résolution 32×32
Tête medium	<code>Conv2d(64, 3, 1×1) + Sigmoid → $\hat{x}_m$ (B,3,32,32)</code>	Reconstruction 32px
Niveau 3 (fine)	<code>ConvTranspose2d(64→32, 4×4, stride=2) → (B,32,64,64) + BN + GELU</code>	Résolution 64×64
Tête fine	<code>Conv2d(32, 3, 1×1) + Sigmoid → $\hat{x}_f$ (B,3,64,64)</code>	Reconstruction 64px
Skip connections	Features du niveau N réinjectées au niveau N+1	Préserve les détails fins

4.4 Préparation des cibles multi-échelle

L'image originale (224×224) doit être redimensionnée à chaque résolution cible avant le calcul de la loss.

4.5 Critères de validation

- PSNR (Peak Signal-to-Noise Ratio) sur $\hat{x}_{\text{fine}}$ > 25 dB sur le val set d'authentiques
- SSIM sur $\hat{x}_{\text{fine}}$ > 0.80 sur le val set d'authentiques
- Reconstruction visuelle cohérente : les textes et zones de sécurité doivent être lisibles dans $\hat{x}_{\text{fine}}$
- Le ratio erreur_forgé / erreur_authentique > 1.5 (les forgeries sont moins bien reconstruites)

05

Reconstruction Loss

MSE + SSIM multi-échelle avec β -KL annealing

5.1 Pourquoi combiner MSE et SSIM

La Mean Squared Error (MSE) pénalise les erreurs pixel-wise brutes. Elle est simple et efficace mais a une limitation : elle traite tous les pixels indépendamment. Une erreur de 0.1 sur un pixel de fond et une erreur de 0.1 sur un pixel de texte sont traitées identiquement, alors que l'erreur sur le texte est bien plus significative forensiquement.

SSIM (Structural Similarity Index) mesure la similarité structurelle entre deux images en combinant trois composantes : luminance (L), contraste (C) et structure (S). Il est plus corrélé à la perception visuelle humaine et détecte mieux les altérations des structures locales (bords de texte, patterns de sécurité).

5.2 Définition formelle de la loss totale

$$L_{\text{total}} = L_{\text{recon}} + \beta(t) \times L_{\text{KL}}$$

$$L_{\text{recon}} = \lambda_c \times L_c + \lambda_m \times L_m + \lambda_f \times L_f$$

$$L_x = \alpha \times \text{MSE}(x, \hat{x}) + \gamma \times (1 - \text{SSIM}(x, \hat{x}))$$

Poids : $\alpha=0.8$, $\gamma=0.2$, $\lambda_c=0.1$, $\lambda_m=0.3$, $\lambda_f=0.6$

$\beta(t)$: annealing 0→4 sur 20 epochs, puis $\beta=4$ fixe

Paramètre	Valeur / Spécification	Justification
L_MSE	$\text{mean}((x - \hat{x})^2)$ pixel-wise	Pénalise les erreurs brutes
L_SSIM	$1 - \text{SSIM}(x, \hat{x})$	Capture structure et contraste
α (poids MSE)	0.8	MSE dominant — ablation sur {0.5,0.7,0.8,1.0}
γ (poids SSIM)	0.2	SSIM complémentaire
λ_{coarse}	0.1	Faible : résolution grossière
λ_{medium}	0.3	Intermédiaire
λ_{fine}	0.6	Dominant : résolution cible principale
β KL (final)	4.0	β -VAE — ablation sur {1,2,4,8}
Optimiseur	AdamW, lr=1e-4, wd=1e-4	Pour Projection Head + Decoder uniquement
Batch size	32	Compromis mémoire GPU / stabilité

Paramètre	Valeur / Spécification	Justification
Epochs	100 + early stopping patience=15	Arrêt si val loss ne diminue pas

5.4 Protocole d'entraînement en 3 phases

Phase	Epochs	β	Objectif	Diagnostic
Phase 1 : Reconstruction pure	1–20	0 (KL désactivée)	Vérifier convergence du décodeur	val loss doit décroître régulièrement
Phase 2 : KL warm-up	21–50	0 $\rightarrow$ 4 (linéaire)	Structurer l'espace latent progressivement	Surveiller le posterior collapse
Phase 3 : Entraînement complet	51–100	4 fixe	Affiner la représentation sparse	Active ratio stable entre 0.3 et 0.7

⚠ Attention

Les images forgées de FMIDV-2022 ne doivent JAMAIS apparaître dans le dataloader d'entraînement. Créer des Dataset classes séparées pour MIDV-2020 (train/val) et FMIDV-2022 (test only) avec des checks explicites.

06

Multi-Scale Anomaly Heatmap

Localisation pixel-level des régions forgées sans supervision

6.1 Principe de la localisation

Après entraînement, le modèle reconstruit bien les documents authentiques mais échoue sur les zones forgées. L'idée est simple : si un pixel (ou une zone) est forgé, l'erreur de reconstruction sera plus élevée à cet endroit. En calculant l'erreur de reconstruction pixel-par-pixel (au lieu d'une erreur globale), on obtient une carte spatiale qui localise les régions suspectes.

Le multi-échelle apporte un avantage critique : une forgerie 16×16 sera mieux visible dans l'erreur à résolution 16×16 (où elle occupe une fraction importante de l'image) que dans l'erreur à résolution 64×64 (où elle occupe seulement 1/16 de l'espace).

6.2 Construction détaillée de la heatmap

Chaque erreur d'échelle est calculée comme la moyenne sur les 3 canaux RGB de l'erreur au carré pixel-wise, puis upscalée à la résolution commune 64×64 :

```
E_c(i,j) = mean_c((x_c(c,i,j) - x_hat_c(c,i,j))2) → upscale → (64,64)
E_m(i,j) = mean_c((x_m(c,i,j) - x_hat_m(c,i,j))2) → upscale → (64,64)
E_f(i,j) = mean_c((x_f(c,i,j) - x_hat_f(c,i,j))2)
H(i,j) = 0.2 × E_c(i,j) + 0.3 × E_m(i,j) + 0.5 × E_f(i,j)
```

6.3 Implémentation de la heatmap

```
import torch.nn.functional as F
from scipy.ndimage import gaussian_filter

def compute_multiscale_heatmap(model, image, train_stats):
    """
    Génère la heatmap d'anomalie pour une image.
    train_stats : dict avec 'mean' et 'std' des heatmaps sur val
    authentiques.
    """
    model.eval()
    with torch.no_grad():
        # Forward pass complet
        patch_tokens, cls_token = extract_dinov2_features(dinov2, image)
        mu, logvar = projection_head(patch_tokens)
        z_sparse, kl, mask = sparse_latent(mu, logvar)
        x_hat_c, x_hat_m, x_hat_f = decoder(z_sparse)

    # Cibles multi-échelle
    x_c, x_m, x_f = prepare_multiscale_targets(image)
```

```

# Erreurs pixelwise : mean sur canaux → (B, 1, H, W)
E_c = ((x_c - x_hat_c)**2).mean(dim=1, keepdim=True) # (B,1,16,16)
E_m = ((x_m - x_hat_m)**2).mean(dim=1, keepdim=True) # (B,1,32,32)
E_f = ((x_f - x_hat_f)**2).mean(dim=1, keepdim=True) # (B,1,64,64)

# Upscaling des erreurs grossières vers 64x64
E_c_up = F.interpolate(E_c, size=(64,64), mode='bilinear',
align_corners=False)
E_m_up = F.interpolate(E_m, size=(64,64), mode='bilinear',
align_corners=False)

# Fusion pondérée
H = 0.2*E_c_up + 0.3*E_m_up + 0.5*E_f # (B, 1, 64, 64)

# Normalisation par les stats du val authentique
H_norm = (H - train_stats['mean']) / (train_stats['std'] + 1e-8)

# Lissage gaussien pour réduire le bruit
H_np = H_norm.squeeze().cpu().numpy()
H_smooth = gaussian_filter(H_np, sigma=1.5)

return H_smooth # numpy array (64, 64)

def visualize_heatmap(original_image, heatmap, threshold_percentile=95):
    """Superpose la heatmap sur l'image originale."""
    import matplotlib.pyplot as plt
    import cv2

    # Normaliser heatmap en [0,1] pour visualisation
    hm = (heatmap - heatmap.min()) / (heatmap.max() - heatmap.min() + 1e-
8)

    # Seuillage binaire
    threshold = np.percentile(hm, threshold_percentile)
    binary_mask = (hm > threshold).astype(np.uint8)

    # Redimensionner vers résolution originale pour visualisation
    hm_resized = cv2.resize(hm, (224, 224),
interpolation=cv2.INTER_LINEAR)
    mask_resized = cv2.resize(binary_mask, (224, 224))

    return hm_resized, mask_resized

```

6.4 Calcul des stats de normalisation

Les statistiques de normalisation (μ , σ) de la heatmap doivent être calculées uniquement sur les documents authentiques du val set, avant l'évaluation sur les forgeries :

```

def compute_heatmap_stats(model, val_loader_authentic):
    """Calcule mean et std des heatmaps brutes sur le val authentique."""
    all_heatmaps = []
    for batch in val_loader_authentic:

```



```

        H = compute_multiscale_heatmap(model, batch['image'],
train_stats=None)
        all_heatmaps.append(H)
        hm_stack = np.stack(all_heatmaps)
        return {'mean': hm_stack.mean(), 'std': hm_stack.std()}

```

6.5 Métriques d'évaluation de la localisation

Métrique	Calcul	Cible	Dataset
IoU	$\frac{ \text{prédiction} \cap \text{GT} }{ \text{prédiction} \cup \text{GT} }$	≥ 0.35 (large forgeries)	FMIDV-2022
Pixel AUC-ROC	Courbe ROC au niveau pixel	≥ 0.75	FMIDV-2022
Average Precision	Aire sous la courbe précision-rappel pixel	≥ 0.60	FMIDV-2022
IoU par granularité	IoU séparé 16px / 32px / 64px	Croissant avec taille	FMIDV-2022

07

Score d'Anomalie Composite

Agrégation multi-signal → classification finale authentique / forgé

7.1 Pourquoi un score composite

Chaque composante du pipeline produit un signal d'anomalie partiel. L'erreur de reconstruction capture les déviations globales. La KL divergence capture les déviations dans l'espace latent. La heatmap capture les déviations localisées. La norme latente capture l'intensité des activations. Aucun signal seul n'est suffisant pour tous les types de forgeries : combiner les quatre donne un score plus robuste.

7.2 Définition des composantes et du score

```

S_recon = L_recon(x, x^_f)                (erreur
reconstruction fine)
S_kl     = KL(q(z|x) || p(z))             (déviatio
distribution latente)
S_heatmap= percentile_95(H_smooth)        (pic heatmap
normalisé)
S_latent = ||z_sparse||_2 / K              (norme moyenne
dime actives)

S_norm_i = (S_i - μ_i) / σ_i pour i ∈ {recon, kl, heatmap,
latent}
S_final  = 0.4×S_norm_recon + 0.2×S_norm_kl
          + 0.3×S_norm_heatmap + 0.1×S_norm_latent

```

Paramètre	Valeur / Spécification	Justification
S_recon	MSE(x_f , $\hat{x}_f$) moyen sur l'image	Signal principal d'anomalie spatiale
S_kl	KL divergence — calculée à l'étape 3	Déviatio de la prior $N(0, I)$
S_heatmap	Percentile 95 de H_smooth	Intensité de la zone la plus suspecte
S_latent	$\ z_sparse\ _2 / K$	Activations anormalement intenses
Normalisation	z-score par rapport au val authentique	Comparabilité entre composantes
Poids ($\alpha, \beta, \gamma, \delta$)	(0.4, 0.2, 0.3, 0.1) — ablation requise	Optimisés sur val set
Seuil θ	Percentile 95 du score sur val authentiques	Recalibré par dataset
Décision	$S > \theta \rightarrow \text{FORGÉ}$; $S \leq \theta \rightarrow \text{AUTHENTIQUE}$	Ajustable selon le coût FP/FN

7.3 Implémentation complète

```
class AnomalyScorer:
    def __init__(self, weights=(0.4, 0.2, 0.3, 0.1)):
        self.w_recon, self.w_kl, self.w_hm, self.w_lat = weights
        self.stats = {} # Statistiques de normalisation

    def fit(self, model, val_loader_authentic):
        """Calibre le scorer sur le val set authentique."""
        scores = {'recon':[], 'kl':[], 'hm':[], 'lat':[]}
        for batch in val_loader_authentic:
            s = self._compute_raw_scores(model, batch['image'])
            for k in scores: scores[k].append(s[k])

        # Calcul des stats pour normalisation
        for k in scores:
            vals = np.array(scores[k])
            self.stats[k] = {'mean': vals.mean(), 'std': vals.std() + 1e-8}

        # Seuil  $\theta$  = percentile 95 du score composite sur les authentiques
        composite = [self._composite(self._normalize(s)) for s in zip(*scores.values())]
        self.threshold = np.percentile(composite, 95)
        print(f'Threshold calibré :  $\theta$  = {self.threshold:.4f}')

    def predict(self, model, image):
        raw = self._compute_raw_scores(model, image)
        norm = self._normalize(raw)
        score = self._composite(norm)
        label = 'FORGÉ' if score > self.threshold else 'AUTHENTIQUE'
        return score, label

    def _composite(self, norm_scores):
        return (self.w_recon * norm_scores['recon'] +
                self.w_kl * norm_scores['kl'] +
                self.w_hm * norm_scores['hm'] +
                self.w_lat * norm_scores['lat'])
```

7.4 Ablation study — obligatoire pour publication

Pour justifier chaque composante dans le papier, mesurer l'AUC-ROC sur FMIDV-2022 pour chaque configuration :

Configuration	Composantes utilisées	AUC attendu	Objectif
Baseline	S_recon seul	~0.68–0.75	Ligne de base
+ KL	S_recon + S_kl	~0.72–0.78	Impact de la contrainte latente
+ Heatmap	S_recon + S_heatmap	~0.78–0.85	Impact de la localisation
Complet	Tous les 4 signaux	Max	Notre méthode finale

7.5 Évaluation cross-dataset

Dataset	Mode	Seuil θ	Métrique principale	Cible
FMIDV-2022	Évaluation principale	Calibré sur MIDV val	AUC-ROC + F1	AUC > 0.85
FantasyID	Zero-shot transfer	Recalibré sur FantasyID val authentiques	F1 + MCC	Compétitif vs EdgeDoc
SIDTD	Zero-shot transfer	Recalibré sur SIDTD val authentiques	AUC-ROC	AUC > 0.70
Données propriétaires	Test conditions réelles	Recalibré sur données propriétaires val	Précision + Rappel	Argument applicatif

✓ Bonne pratique

Pour la recalibration du seuil θ sur un nouveau dataset : utiliser uniquement les documents authentiques du nouveau dataset (jamais les forgeries) pour calculer les nouvelles statistiques de normalisation et le nouveau percentile 95. Cela est valide car notre modèle est non supervisé.

Stack technique et environnement

Dépendances Python

```
# Installation complète
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
pip install kornia                # SSIM et losses avancées
pip install wandb                 # Suivi des expériences
pip install scikit-learn          # Métriques AUC-ROC, IoU
pip install opencv-python         # Visualisation heatmap
pip install matplotlib seaborn
pip install scipy                 # Gaussian filter pour heatmap
pip install timm                  # Optionnel : modèles ViT alternatifs

# DINOv2 via torch.hub (pas besoin de pip install séparé)
import torch
dinov2 = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14')
```

Suivi des expériences avec Weights & Biases

```
import wandb

wandb.init(project='sparse-vae-forgery', config={
    'latent_dim': 64, 'k': 16, 'beta': 4,
    'alpha': 0.8, 'gamma': 0.2, 'lr': 1e-4
})

# À chaque epoch – logger toutes les métriques
wandb.log({
    'loss/total': l_total,
    'loss/recon': l_recon,
    'loss/kl': l_kl,
    'latent/active_ratio': active_ratio,
    'latent/sparsity_rate': sparsity_rate,
    'eval/auc_roc': auc_roc,
    'eval/iou_large': iou_large,
    'heatmap': wandb.Image(heatmap_example), # Visualisation
})
```

Organisation des fichiers recommandée

```
sparse_vae_forgery/
├── data/
│   ├── midv2020/          # Images authentiques (train/val/test)
│   ├── fmidv2022/         # Forgeries FMIDV (test only)
│   ├── fantasyid/         # FantasyID (test only)
│   └── sidtd/             # SIDTD (test only)
└── models/
```

```
|   ├── projection_head.py
|   ├── sparse_latent.py
|   ├── multiscale_decoder.py
|   └── full_model.py
├── losses/
|   └── total_loss.py
├── evaluation/
|   ├── heatmap.py
|   ├── anomaly_scorer.py
|   └── metrics.py
├── train.py
├── evaluate.py
└── configs/
    └── default.yaml      # Hyperparamètres
```

Reproductibilité — obligatoire pour publication

```
import torch, numpy as np, random

def set_seed(seed=42):
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    np.random.seed(seed)
    random.seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)  # Appeler avant tout — résultats reproductibles garantis
```